Введение в алгоритмы сортировки

Insert Sort, Select Sort, Quick Sort

Прикладная проблема

Пусть у нас есть некоторый алгоритм, который моделирует процесс развития популяции. Для формирования нового поколения в популяции, необходимо отобрать родителей с хорошей приспособленностью к среде.

A - 0.3	B - 0.6	C - 0.25	D - 0.8	E - 0.27	A - 0.99	…

Возможно ли произвести отбор хромосом эффективно?

Прикладная проблема

Пусть у нас есть некоторый алгоритм, который моделирует процесс развития популяции. Для формирования нового поколения в популяции, необходимо отобрать родителей с хорошей приспособленностью к среде.

A - 0.3	B - 0.6	C - 0.25	D - 0.8	E - 0.27	A - 0.99	…

Возможно ли произвести отбор хромосом эффективно?

A - 0.99	D - 0.8	B - 0.6	A - 0.3	E - 0.27	C - 0.25	0

Прикладная проблема

Пусть у нас есть некоторый алгоритм, который моделирует процесс развития популяции. Для формирования нового поколения в популяции, необходимо отобрать родителей с хорошей приспособленностью к среде.

A - 0.3	B - 0.6	C - 0.25	D - 0.8	E - 0.27	A - 0.99	…

Возможно ли произвести отбор хромосом эффективно?

A - 0.99	D - 0.8	B - 0.6	A - 0.3	E - 0.27	C - 0.25	0

Для чего нужны алгоритмы сортировки

Алгоритм сортировки – это хорошая основа для более сложных алгоритмов. Задача алгоритма сортировки заключается в подготовке данных для других, более эффективных и быстрых алгоритмов.

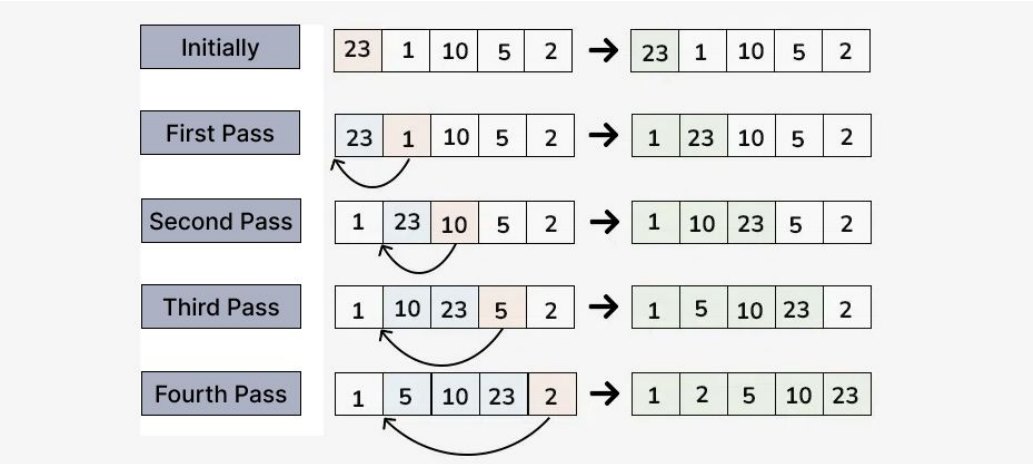
Это могут быть:

- Алгоритмы поиска и вставки;
- Древовидные структуры данных;
- Оптимизация данных на уровне памяти.

Все потому, что многие эффективные алгоритмы требуют данные в уже отсортированном виде. Потому они такие эффективные.

Первая сортировка - Insert Sort

Insert Sort – итеративно вставляет каждый элемент из неотсортированной части списка в его правильное положение в отсортированной части списка.



Insertion Sort Algorithm - GeeksforGeeks

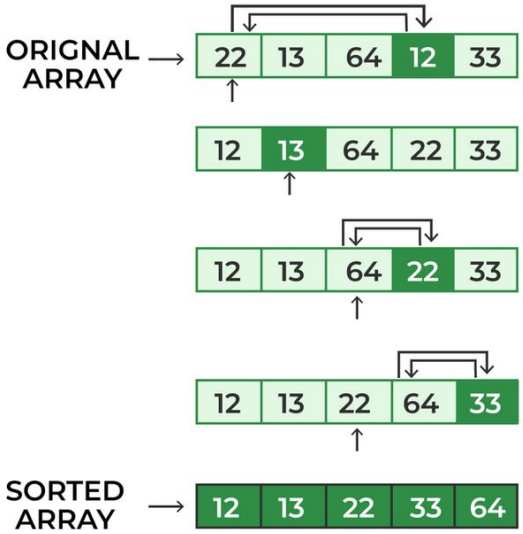
Первая сортировка - Insert Sort

```
void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; ++i) {
        int key = arr[i];
        int j = i - 1;

        /* Move elements of arr[0..i-1], that are
           greater than key, to one position ahead
           of their current position */
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
```

Select sort

Select sort – алгоритм сортировки на основе перестановки (swap).



Итеративно переставляется наименьший (или наибольший) элемент из неотсортированной части с первым элементом из неотсортированной части.

[Selection Sort - GeeksforGeeks]

Select sort

```
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; ++i) {

        // Assume the current position holds
        // the minimum element
        int min_idx = i;

        // Iterate through the unsorted portion
        // to find the actual minimum
        for (int j = i + 1; j < n; ++j) {
            if (arr[j] < arr[min_idx]) {

                // Update min_idx if a smaller element is found
                min_idx = j;
            }
        }

        // Move minimum element to its
        // correct position
        int temp = arr[i];
        arr[i] = arr[min_idx];
        arr[min_idx] = temp;
    }
}
```

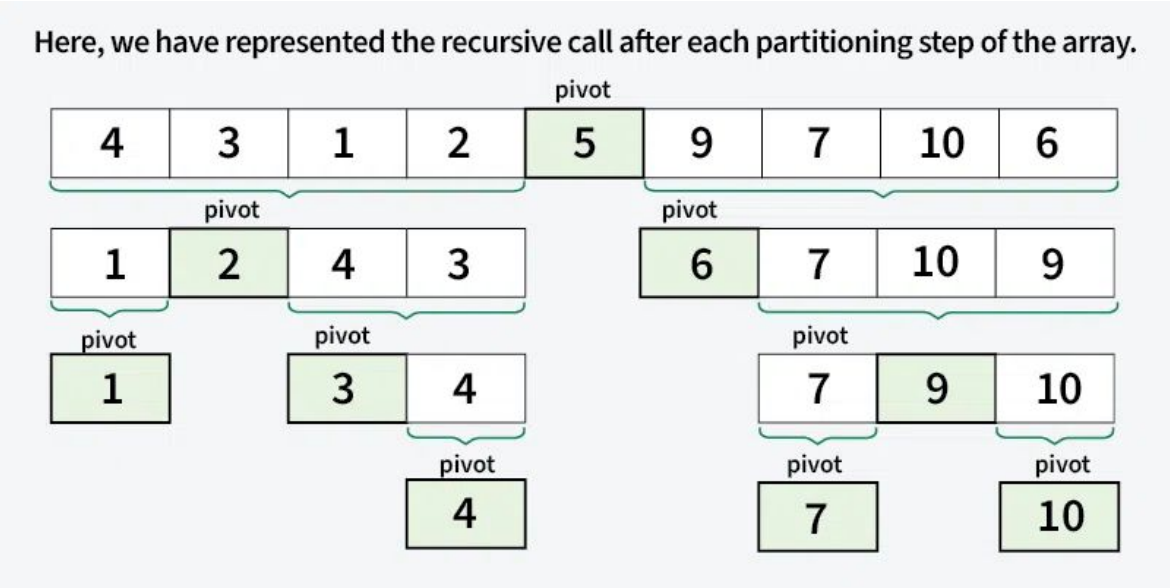
Алгоритмы “Разделяй и властвуй”

Идея алгоритмов “Разделяй и властвуй” состоит в том, чтобы разбить исходную задачу на более мелкие подзадачи, решить подзадачи рекурсивно и, наконец, объединить решения в итоговое решения исходной задачи.

К алгоритмам “Разделяй и властвуй” относятся:

- Алгоритм MergeSort – Исходный массив рекурсивно разделяется на подмассивы пока длина подмассива не будет равна 1, затем подмассивы собираются в исходный в отсортированном виде;
- Алгоритм QuickSort – Выбирается опорный элемент и ставится на правильную позицию, элементы справа и слева от опороного разбиваются на подзадачи и решаются рекурсивно.

Quick Sort



[Quick Sort - GeeksforGeeks]

Quick Sort

```
// The QuickSort function implementation
void quickSort(int arr[], int low, int high) {
    if (low < high) {

        // pi is the partition return index of pivot
        int pi = partition(arr, low, high);

        // recursion calls for smaller elements
        // and greater or equals elements
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

Функции swap(...) и partition(...)

```
void swap(int* a, int* b) {
    int t = *a;
    *a = *b;
    *b = t;
}
```

```
int partition(int arr[], int low, int high) {

    // Choose the pivot
    int pivot = arr[high];

    // Index of smaller element and indicates
    // the right position of pivot found so far
    int i = low - 1;

    // Traverse arr[low..high] and move all smaller
    // elements to the left side. Elements from low to
    // i are smaller after every iteration
    for (int j = low; j <= high - 1; ++j) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    // Move pivot after smaller elements and
    // return its position
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}
```

Домашнее задание

Для Пдо-27: Найти и реализовать алгоритм сортировки слиянием (Merge Sort) на языке программирования C. Исходный массив должен задаваться через ввод в терминале. Дополнительно должны быть предусмотрены проверки на ввод символов (подумайте почему, спрошу). Если вводится символ, то вызывается функция abort().

Для Пд-21: Найти и реализовать алгоритм сортировки слиянием (Merge Sort) на языке программирования 1С. Разработать в конфигурации (**Управляемое приложение**) обработчик ввода сырого массива и его вывода в отсортированной форме.

Защитой кода считается его умелое чтение!!!.